第7章: CRUD実装（一覧表示・新規登録）

> いよいよアプリの中心機能を作ります。この章では CRUD のうち **R（Read＝一覧表示）** と **C（Create＝新規登録）** を実装します。第6章で用意した `lib/books.ts` の関数と、第4章で学んだ `FlatList`・`TextInput` を使います。コードは全行コメント付きで、初めての書き方はすべて解説します。

---

1. この章のゴール

- Supabaseから本のデータを取得し、**`FlatList` で一覧表示**する
- 「読込中」の表示や、本が0件のときの表示も用意する
- **入力フォーム**で新しい本を登録し、保存後に一覧へ戻る

---

2. 書籍一覧画面を作る（Read）

第6章で仮置きした `app/index.tsx` を、本物の一覧画面に作り替えます。少しずつ理解できるよう、まず完成形を示し、その後ポイントを解説します。

> **▼ このコードがやること（先に日本語で）**：Supabase（データの保管庫）から本のデータを取り出し、`FlatList`（たくさんの項目を効率よく縦に並べて表示する部品）で一覧画面を作ります。読み込んでいる間は「ぐるぐる」（`ActivityIndicator`）を出し、本が0件なら案内文を出す、という気配りも入れます。カギになるのは `useFocusEffect`（この画面が表示されるたびに実行されるしくみ）で、これのおかげで登録画面から戻った瞬間に最新の本が並びます。1行ずつの細かい意味はコード内のコメントで説明しているので、まずは「データを取って一覧に並べる画面」という全体像をつかんでください。

コピー

```
// app/index.tsx – 書籍一覧画面（Read）

import { useState, useCallback } from "react";
import { View, Text, FlatList, Pressable, StyleSheet, ActivityIndicator } from "react-native";
import { useRouter, useFocusEffect } from "expo-router";
// 第6章で作った「全件取得」関数
import { fetchBooks } from "../lib/books";
// 第6章で作ったBook型
import { Book } from "../types/book";

export default function HomeScreen() {
  // 画面移動の道具（第4章）
  const router = useRouter();
  // 本のリストをstateで持つ。<Book[]>で型を指定。初期値は空配列
  const [books, setBooks] = useState<Book[]>([]);
  // 読込中かどうか。最初はtrue（読込中）
  const [loading, setLoading] = useState(true);

  // データを読み込む関数。async（待つ処理を含む）
  const load = useCallback(async () => {
    // try ： この中の処理を試す。エラーが出たらcatchへ飛ぶ
    try {
      // 読込開始：ローディング表示をオンに
      setLoading(true);
      // Supabaseから全件取得（結果が返るまで待つ）
      const data = await fetchBooks();
      // 取得したデータをstateにセット → 画面が更新される
      setBooks(data);
    // catch ： tryの中でエラーが起きたらここに来る。eにエラーが入る
    } catch (e) {
      // エラー内容を表示（学習用。本番は画面に出すと親切）
      console.log("読み込みエラー:", e);
    // finally ： 成功・失敗どちらでも最後に必ず実行
    } finally {
      // 読込終了：ローディング表示をオフに
      setLoading(false);
    }
  }, []);

  // useFocusEffect ： 「画面が表示される（フォーカスされる）たび」に実行するExpo Routerのフック
  // 登録画面から戻ってきたときも再取得され、追加した本がすぐ反映される
  useFocusEffect(
    useCallback(() => {
      load();
    }, [load])
  );

  // 読込中はぐるぐる回るインジケータを表示して、ここで描画を終える（早期return）
  if (loading) {
    return (
      <View style={styles.center}>
        {/* ActivityIndicator ： 読込中のぐるぐる */}
        <ActivityIndicator size="large" color="#1e40af" />
      </View>
    );
  }

  return (
    <View style={styles.container}>
      <FlatList
        // 表示するデータ（本の配列）
        data={books}
        // 各本を区別するキー（idを使う）
```

```
        keyExtractor={(item) => item.id}
        // リスト内側の余白と要素間の隙間
        contentContainerStyle={{ padding: 16, gap: 10 }}
        // ListEmptyComponent : dataが空のときに表示する内容
        ListEmptyComponent={
          <Text style={styles.empty}>まだ本が登録されていません。右下の＋から追加しましょう。</Text>
        }
        // renderItem : 1冊分の見た目。itemにその本が入る（第4章のFlatList参照）
        renderItem={({ item }) => (
          <View style={styles.card}>
            <Text style={styles.title}>{item.title}</Text>
            <Text style={styles.author}>著者: {item.author}</Text>
            {/* ステータスを色付きバッジで表示。statusの値で色を変える（下のgetStatusStyle） */}
            <View style={[styles.badge, getStatusStyle(item.status)]}>
              <Text style={styles.badgeText}>{item.status}</Text>
            </View>
          </View>
        )}
      />

      {/* 画面右下に浮かぶ追加ボタン（FAB = Floating Action Button） */}
      <Pressable style={styles.fab} onPress={() => router.push("/new")}>
        <Text style={styles.fabText}>＋</Text>
      </Pressable>
    </View>
  );
}

// ステータスごとに色を変える補助関数。引数statusに応じて背景色を返す
function getStatusStyle(status: string) {
  // 緑系
  if (status === "読了") return { backgroundColor: "#dcfce7" };
  // 青系
  if (status === "読書中") return { backgroundColor: "#dbeafe" };
  // それ以外（未読）は黄系
  return { backgroundColor: "#fef3c7" };
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f8fafc" },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  empty: { textAlign: "center", color: "#94a3b8", marginTop: 40, paddingHorizontal: 20 },
  card: {
    backgroundColor: "#fff",
    borderRadius: 10,
    padding: 14,
    borderWidth: 1,
    borderColor: "#e2e8f0",
  },
  title: { fontSize: 15, fontWeight: "bold", color: "#1e293b" },
  author: { fontSize: 12, color: "#64748b", marginTop: 3 },
  badge: { alignSelf: "flex-start", marginTop: 8, paddingVertical: 3, paddingHorizontal: 10, borderRadius: 20 },
  badgeText: { fontSize: 11, fontWeight: "600", color: "#334155" },
  fab: {
    // position:"absolute" : 通常の流れから外して自由配置
    position: "absolute",
    // 右から20、下から30の位置に固定
    right: 20, bottom: 30,
    // 直径56の円（borderRadiusを半分にすると円になる）
    width: 56, height: 56, borderRadius: 28,
    backgroundColor: "#1e40af",
    justifyContent: "center", alignItems: "center",
    shadowColor: "#000", shadowOpacity: 0.3, shadowRadius: 4, shadowOffset: { width: 0, height: 2 },
    // elevation : Androidの影の深さ（iOSはshadow系で指定）
    elevation: 5,
  },
```

```
  fabText: { color: "#fff", fontSize: 28, lineHeight: 30 },
});
```

2.1 重要ポイントの解説

① `useState<Book[]>([])` の `<Book[]>` `useState` に `<Book[]>`（山カッコで型を指定）を付けて「このstateはBookの配列だ」と明示しています。これは第2章で学んだ**ジェネリクス**（入れ物の型に"中身の型"を渡すしくみ）です。`useState` という入れ物に「中身は `Book[]`（本の配列）ですよ」と教えているわけです。こうすると `books` に対して `item.title` などを書くとき、TypeScriptが補完・チェックしてくれます。初期値の `[]`（空配列）は「最初は本が0冊」という意味です。

② `try / catch / finally` サーバー通信は失敗することがあります（電波が悪いなど）。`try` で試し、失敗したら `catch` でエラーを受け止め、`finally` で「成功でも失敗でも必ずやること（ローディングを消す）」を書きます。これでアプリが固まりません。

> **`try / catch / finally` の流れ:** - `try { ... }` … まずここを実行してみる。 - `catch (e) { ... }` … `try` の中でエラーが起きたら、ここに飛んでくる（ `e` にエラー情報）。 - `finally { ... }` … 成功・失敗どちらでも、最後に必ず実行される。

③ `useFocusEffect` **を使う理由** 第3章で `useEffect`（最初の1回だけ実行）を学びましたが、ここでは `useFocusEffect`（画面が表示されるたびに実行）を使います。理由は「新規登録画面で本を追加して一覧に戻ったとき、**戻った瞬間に再取得**して最新のリストを見せたい」からです。`useEffect` だと最初の1回しか動かず、追加した本がすぐ反映されません。

> **`useCallback` とは？ なぜ `useFocusEffect` と組み合わせるの？**「おまじない」で済ませず、理由を説明します。
>
> まず前提として、Reactのコンポーネント（画面の関数）は、stateが変わるたびに**関数ぜんたいが何度も再実行**されます。すると、その中で定義している `load` のような関数は、**実行のたびに"別物"として作り直されて**しまいます（中身は同じでも、コンピュータから見ると毎回新しい関数）。
>
> ここで困ったことが起きます。`useFocusEffect` は「**渡された関数が"別物"に変わったら、処理をやり直す**」性質があります。もし `load` が毎回作り直されると、`useFocusEffect` は「関数が変わった！」と勘違いし、**何度も無駄に再取得（最悪は無限ループ）**してしまうのです。
>
> これを防ぐのが **`useCallback`（ユーズ・コールバック）** です。「関数を**毎回作り直さず、覚えておく（メモする＝メモ化）**」フックで、第2引数の依存配列（ `[load]` の部分）の中身が変わらない限り、**同じ関数を使い回し**ます。これで `useFocusEffect` が「変わっていない」と正しく認識でき、無駄な再実行が止まります。
>
> まとめると `useFocusEffect(useCallback(() => { load() }, [load]))` は「**画面が表示されるたびに `load` を呼ぶ。ただし `load` は毎回作り直さず使い回す**」という意味です。`useFocusEffect` と `useCallback` がセットで出てくるのは、この「無駄な再実行を防ぐ」ためのお決まりだと理解しておけば十分です。

④ `ListEmptyComponent` `FlatList` に標準で備わる便利機能で、「データが0件のときに表示する内容」を指定できます。「まだ本がありません」という案内を出して、空っぽの寂しい画面を防ぎます。

▼ コードを1つずつ分解して解説

上の `app/index.tsx` には、初めて見る書き方がいくつも入っています。コードを意味の「塊（かたまり）」に分けて、**塊ごとにもう一度コードを載せながら**、初心者向けにていねいに解説します。一度で全部覚えなくて大丈夫です。

解説1: import（必要な道具を読み込む）

コピー

```
import { useState, useCallback } from "react";
import { View, Text, FlatList, Pressable, StyleSheet, ActivityIndicator } from "react-native";
import { useRouter, useFocusEffect } from "expo-router";
// 第6章で作った「全件取得」関数
import { fetchBooks } from "../lib/books";
// 第6章で作ったBook型
import { Book } from "../types/book";
```

- `import { A, B } from "..."` は「`...` というファイル（ライブラリ）から、`A` と `B` という道具だけを取り出して使う」という意味です。
- 1行目は React の**フック**（`useState` ＝値を覚える、`useCallback` ＝関数を覚える）を読み込んでいます。
- 2行目は React Native の**部品**（`View` ＝箱、`Text` ＝文字、`FlatList` ＝一覧、`Pressable` ＝押せる領域、`StyleSheet` ＝見た目の定義、`ActivityIndicator` ＝ぐるぐる）です。
- 4・5行目は、第6章で**自分で作ったファイル**から `fetchBooks`（取得関数）と `Book`（型）を読み込んでいます。`../` は「1つ上のフォルダ」という意味です。

> **用語: フック（Hook）** … `use` で始まる React の特別な関数のこと。コンポーネント（画面の関数）の中で「値を覚える」「処理を実行する」などの機能を追加します。

解説2: state を用意する（画面が覚えておく値）

コピー

```
// 画面移動の道具（第4章）
const router = useRouter();
// 本のリストをstateで持つ。<Book[]>で型を指定。初期値は空配列
const [books, setBooks] = useState<Book[]>([]);
// 読込中かどうか。最初はtrue（読込中）
const [loading, setLoading] = useState(true);
```

- `useRouter()` は「別の画面へ移動する」ための道具を `router` に入れています（あとで `router.push("/new")` で登録画面へ飛びます）。

- `const [今の値, 変える関数] = useState(初期値)` は、画面が覚えておく値（state）を作る決まり文句です。
- `books` には本の一覧（`Book` の配列）を入れます。`<Book[]>` で「中身は本の配列」と型を指定し、初期値 `[]` は「最初は0冊」という意味です。
- `loading` は「いま読込中か」を表す `true` / `false` の値です。最初は `true`（読込中）から始めます。

> **用語: state（ステート）** … コンポーネントが内部で覚えておく「変化する値」。`setBooks` のような更新関数で書き換えると、画面が自動で描き直されます。

解説3: データを読み込む関数 `load`

コピー

```
const load = useCallback(async () => {
  // try : この中の処理を試す。エラーが出たらcatchへ飛ぶ
  try {
    // 読込開始：ローディング表示をオンに
    setLoading(true);
    // Supabaseから全件取得（結果が返るまで待つ）
    const data = await fetchBooks();
    // 取得したデータをstateにセット → 画面が更新される
    setBooks(data);
  // catch : tryの中でエラーが起きたらここに来る。eにエラーが入る
  } catch (e) {
    // エラー内容を表示（学習用。本番は画面に出すと親切）
    console.log("読み込みエラー:", e);
  // finally : 成功・失敗どちらでも最後に必ず実行
  } finally {
    // 読込終了：ローディング表示をオフに
    setLoading(false);
  }
}, []);
```

- `load` は「本を取得して画面に反映する」関数です。`async`（エイシンク）は「**待つ処理を含む関数**」の印で、中で `await`（待つ）が使えます。
- `await fetchBooks()` は「Supabaseからデータが返ってくるまで待ってから、次の行へ進む」という意味です。通信は一瞬で終わらないため「待つ」必要があります。
- `try / catch / finally` で通信の失敗に備えます（`try` で試し、失敗したら `catch`、最後に必ず `finally`）。
- 全体を `useCallback(..., [])` で包んでいるのは、**この関数を毎回作り直さず使い回す**ためです（理由は解説4で詳しく説明します）。

> **用語: async / await** … `async` を付けた関数の中で `await` を書くと、「時間のかかる処理（通信など）が終わるまで待ってから次へ進む」ようにできます。画面を固めずに通信を扱うための書き方です。

解説4: `useFocusEffect` で画面表示のたびに再取得

コピー

```
useFocusEffect(
  useCallback(() => {
    load();
  }, [load])
);
```

- `useFocusEffect` は Expo Router のフックで、「**この画面が表示される（フォーカスされる）たびに、中の処理を実行する**」働きをします。
- そのおかげで、登録画面で本を追加して**一覧に戻ってきた瞬間に `load()` が再び動き**、追加した本がすぐ反映されます。第3章の `useEffect`（最初の1回だけ）では、戻ってきても再取得されません。
- 中身を `useCallback(() => {...}, [load])` で包むのは、**渡す関数を毎回作り直さないようにするため**です。作り直すと `useFocusEffect` が「関数が変わった」と勘違いし、無駄に何度も再実行してしまいます。

> **用語: useCallback（ユーズ・コールバック）** … 関数を「毎回作り直さず覚えておく（メモ化する）」フック。第2引数の `[load]` の中身が変わらない限り、同じ関数を使い回します。`useFocusEffect` の無駄な再実行を防ぐためにセットで使われます。

解説5: 読込中は早期 return でぐるぐるを表示

コピー

```
if (loading) {
  return (
    <View style={styles.center}>
      {/* ActivityIndicator : 読込中のぐるぐる */}
      <ActivityIndicator size="large" color="#1e40af" />
    </View>
  );
}
```

- `if (loading) { return (...) }` は「もし読込中なら、ぐるぐる画面を返して**ここで関数を終わらせる**」という書き方です。この「条件を満たしたら途中で `return` して抜ける」やり方を「**早期 return（early return）**」と呼びます。

- 早期 return で抜けた場合、下の一覧（`FlatList`）の部分までは進みません。データがまだ無いうちは一覧を作らず、ぐるぐるだけを見せます。
- `ActivityIndicator` は「読込中のくるくる回るマーク」です。`size="large"` で大きめ、`color` で色を指定しています。

> **用語: 早期 return（early return）** … 関数の途中で条件に応じて `return` し、その先の処理を実行せずに抜けるテクニック。「読込中ならここで終わり」のように、画面を出し分けるのに使います。

解説6: `FlatList` で一覧を表示する

コピー

```
<FlatList
  // 表示するデータ（本の配列）
  data={books}
  // 各本を区別するキー（idを使う）
  keyExtractor={(item) => item.id}
  // リスト内側の余白と要素間の隙間
  contentContainerStyle={{ padding: 16, gap: 10 }}
  // ListEmptyComponent : dataが空のときに表示する内容
  ListEmptyComponent={
    <Text style={styles.empty}>まだ本が登録されていません。右下の＋から追加しましょう。</Text>
  }
  // renderItem : 1冊分の見た目。itemにその本が入る（第4章のFlatList参照）
  renderItem={({ item }) => (
    <View style={styles.card}>
      <Text style={styles.title}>{item.title}</Text>
      <Text style={styles.author}>著者: {item.author}</Text>
      {/* ステータスを色付きバッジで表示。statusの値で色を変える（下のgetStatusStyle） */}
      <View style={[styles.badge, getStatusStyle(item.status)]}>
        <Text style={styles.badgeText}>{item.status}</Text>
      </View>
    </View>
  )}
/>
```

- `FlatList` は「たくさんの項目を効率よく縦に並べる」部品です。画面に映る分だけを描くので、件数が多くても軽快に動きます。
- `data={books}` で「並べたいデータ（本の配列）」を渡します。`keyExtractor` は各項目を区別する目印で、ここでは本の `id` を使っています。
- `ListEmptyComponent` は「データが0件のときだけ表示する内容」です。空っぽの寂しい画面を防ぎ、追加をうながします。
- `renderItem={({ item }) => (...)}` は「1冊分をどう表示するか」の指定です。`item` にその本が入り、タイトル・著者・ステータスのバッジを組み立てています。

> **用語: keyExtractor（キーエクストラクター）** … `FlatList` の各項目に「他と区別する目印（キー）」を与える設定。React がリストの差分を正しく検出するために使います。一意な `id` を渡すのが鉄則です。

解説7: 右下に浮かぶ追加ボタン（FAB）

コピー

```
{/* 画面右下に浮かぶ追加ボタン（FAB = Floating Action Button） */}
<Pressable style={styles.fab} onPress={() => router.push("/new")}>
  <Text style={styles.fabText}>＋</Text>
</Pressable>
```

- `Pressable` は「押せる領域」を作る部品で、`onPress` に「押されたときの処理」を渡します。
- `onPress={() => router.push("/new")}` は「押したら `/new`（登録画面）へ移動する」という意味です。`router.push` は新しい画面を上に重ねて表示します。
- `style={styles.fab}` の中身（下の `styles`）で、右下に固定して丸く浮かせています。この「画面の上に浮かぶ丸い追加ボタン」を **FAB（Floating Action Button）** と呼びます。

> **用語: FAB（Floating Action Button）** … 画面の右下などに浮かぶ円形の主要アクションボタン。「新規追加」によく使われる、スマホアプリで定番のUIです。

解説8: 補助関数 `getStatusStyle`

コピー

```
// ステータスごとに色を変える補助関数。引数statusに応じて背景色を返す
function getStatusStyle(status: string) {
  // 緑系
  if (status === "読了") return { backgroundColor: "#dcfce7" };
  // 青系
  if (status === "読書中") return { backgroundColor: "#dbeafe" };
  // それ以外（未読）は黄系
  return { backgroundColor: "#fef3c7" };
}
```

- `getStatusStyle` は「ステータスの文字に応じて、バッジの背景色（スタイル）を返す」だけの小さな関数です。
- `if (status === "読了") return {...}` のように、上から順に一致するものを探し、一致したらその色のオブジェクトを `return` します。
- どれにも一致しなければ、最後の行の黄系（未読の色）を返します。これを解説6の `getStatusStyle(item.status)` で呼び出し、本ごとに色を変えています。

> **用語: 補助関数（ヘルパー関数）** … コンポーネントの外に切り出した、計算や変換を担当する小さな関数。見た目のロジックを分けることで、`FlatList` 本体のコードが読みやすくなります。

解説9: `styles`（見た目の定義）

コピー

```
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f8fafc" },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  empty: { textAlign: "center", color: "#94a3b8", marginTop: 40, paddingHorizontal: 20 },
  card: {
    backgroundColor: "#fff",
    borderRadius: 10,
    padding: 14,
    borderWidth: 1,
    borderColor: "#e2e8f0",
  },
  title: { fontSize: 15, fontWeight: "bold", color: "#1e293b" },
  author: { fontSize: 12, color: "#64748b", marginTop: 3 },
  badge: { alignSelf: "flex-start", marginTop: 8, paddingVertical: 3, paddingHorizontal: 10, borderRadius: 20 },
  badgeText: { fontSize: 11, fontWeight: "600", color: "#334155" },
  fab: {
    // position:"absolute" : 通常の流れから外して自由配置
    position: "absolute",
    // 右から20、下から30の位置に固定
    right: 20, bottom: 30,
    // 直径56の円（borderRadiusを半分にすると円になる）
    width: 56, height: 56, borderRadius: 28,
    backgroundColor: "#1e40af",
    justifyContent: "center", alignItems: "center",
    shadowColor: "#000", shadowOpacity: 0.3, shadowRadius: 4, shadowOffset: { width: 0, height: 2 },
    // elevation : Androidの影の深さ（iOSはshadow系で指定）
    elevation: 5,
  },
  fabText: { color: "#fff", fontSize: 28, lineHeight: 30 },
});
```

- `StyleSheet.create({...})` は「画面の見た目（色・余白・配置など）をまとめて定義する」しくみです。CSSに似ていますが、React Native 用の書き方です。
- `container` や `card` などの名前ごとにスタイルをまとめておき、`style={styles.card}` のように名前で呼び出して使います。
- `flex: 1` は「使える空間いっぱいに広がる」、`justifyContent` / `alignItems` は「中身を中央寄せにする」設定です。`center` スタイルでぐるぐるを画面中央に置いています。
- `fab` では `position: "absolute"` で右下に固定し、`width` / `height` / `borderRadius` で円形にしています（`borderRadius` を縦横の半分にすると正円になります）。

> **用語: StyleSheet（スタイルシート）** … React Native で見た目を定義する道具。`StyleSheet.create` で作ったスタイルは名前で使い回せ、書き間違いも検出しやすくなります。

---

3. 新規登録フォームを作る（Create）

第6章で仮置きした `app/new.tsx` を、本物の入力フォームに作り替えます。

> **▼ このコードがやること（先に日本語で）**：新しい本を登録するための入力フォームを作ります。タイトル・著者・ステータス・メモの各入力欄を `TextInput` で用意し、入力された文字を `useState`（画面が覚えておく値）で1つずつ管理します。保存ボタンを押すと、まず「タイトルと著者が空でないか」をチェック（バリデーション）し、問題なければ `createBook`（第6章で作った追加関数）でデータを保存してから一覧画面へ戻ります。保存中はボタンを押せなくして二重登録を防ぐ工夫も入っています。各行の細かい意味はコード内のコメントにあるので、ここでは「入力 → チェック → 保存 → 戻る」という流れをつかんでおけば十分です。

コピー

```
// app/new.tsx – 新規登録画面（Create）

import { useState } from "react";
import { View, Text, TextInput, Pressable, StyleSheet, ScrollView, Alert } from "react-native";
import { useRouter } from "expo-router";
// 第6章で作った「追加」関数
import { createBook } from "../lib/books";

// ステータスの選択肢を配列で用意（ボタンを並べるのに使う）
const STATUS_OPTIONS = ["未読", "読書中", "読了"];

export default function NewBookScreen() {
  const router = useRouter();

  // 入力欄ごとにstateを用意する（第4章の制御コンポーネント）
  // タイトル（最初は空文字）
  const [title, setTitle] = useState("");
  // 著者
  const [author, setAuthor] = useState("");
  // ステータス（初期値は未読）
  const [status, setStatus] = useState("未読");
  // メモ
  const [memo, setMemo] = useState("");
  // 保存処理中かどうか（連打防止に使う）
  const [saving, setSaving] = useState(false);

  // 保存ボタンを押したときの処理
  const handleSave = async () => {
    // 入力チェック（バリデーション）：タイトルと著者は必須
    // .trim() ：文字列の前後の空白を除く。空白だけの入力を「未入力」とみなすため
    if (title.trim() === "" || author.trim() === "") {
      // Alert.alert(タイトル, 本文) ：警告ダイアログを出す
      Alert.alert("入力エラー", "タイトルと著者は必須です");
      // 処理を中断（保存しない）
      return;
    }

    try {
      // 保存開始（ボタンを無効化して二重送信を防ぐ）
      setSaving(true);
      // createBook(...) ：第6章の追加関数。入力値をオブジェクトにまとめて渡す
      await createBook({
        // 前後空白を除いて保存
        title: title.trim(),
        author: author.trim(),
        status: status,
        // メモが空ならnull、あれば値を保存（三項演算子）
        memo: memo.trim() === "" ? null : memo.trim(),
      });
      // 成功したら一覧画面へ戻る。一覧側のuseFocusEffectが再取得し、追加した本が表示される
      router.back();
    } catch (e) {
      Alert.alert("保存に失敗しました", "通信状況を確認してもう一度お試しください");
      console.log("保存エラー:", e);
    } finally {
      // 成功・失敗どちらでも保存状態を解除
      setSaving(false);
    }
  };

  return (
    // ScrollView ：入力欄が多くキーボードで隠れるのを防ぐため、スクロール可能にする
    <ScrollView style={styles.container} contentContainerStyle={{ padding: 20, gap: 18 }}>
      {/* タイトル入力 */}
```

```
      <View>
        <Text style={styles.label}>タイトル *</Text>
        <TextInput
          style={styles.input}
          placeholder="書籍のタイトルを入力"
          // stateと結びつける
          value={title}
          // 入力が変わるたびsetTitleで更新（setTitle(text)の短縮形）
          onChangeText={setTitle}
        />
      </View>

      {/* 著者入力 */}
      <View>
        <Text style={styles.label}>著者 *</Text>
        <TextInput
          style={styles.input}
          placeholder="著者名を入力"
          value={author}
          onChangeText={setAuthor}
        />
      </View>

      {/* ステータス選択（3つのボタンから1つ選ぶ） */}
      <View>
        <Text style={styles.label}>ステータス</Text>
        <View style={styles.statusRow}>
          {/* STATUS_OPTIONS配列をmapでボタンに変換（第2章のmap） */}
          {STATUS_OPTIONS.map((option) => (
            <Pressable
              // map で並べる要素には必ずkeyが必要
              key={option}
              // style に配列を渡すと複数スタイルを合成できる。選択中なら強調スタイルを追加
              style={[styles.statusButton, status === option && styles.statusButtonActive]}
              // 押したらそのステータスを選択
              onPress={() => setStatus(option)}
            >
              <Text style={[styles.statusText, status === option && styles.statusTextActive]}>
                {option}
              </Text>
            </Pressable>
          ))}
        </View>
      </View>

      {/* メモ入力（複数行） */}
      <View>
        <Text style={styles.label}>メモ</Text>
        <TextInput
          style={[styles.input, styles.textarea]}
          placeholder="感想や覚え書きなど"
          value={memo}
          onChangeText={setMemo}
          // multiline : 複数行入力を許可する
          multiline
          // 表示上の高さの目安（4行分）
          numberOfLines={4}
        />
      </View>

      {/* 保存ボタン。saving中は無効化＆文言変更 */}
      <Pressable
        style={[styles.saveButton, saving && styles.saveButtonDisabled]}
        onPress={handleSave}
        // disabled : trueの間は押せなくする（二重送信防止）
        disabled={saving}
      >
```

```
        <Text style={styles.saveButtonText}>{saving ? "保存中..." : "保存する"}</Text>
      </Pressable>
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f8fafc" },
  label: { fontSize: 13, fontWeight: "600", color: "#334155", marginBottom: 6 },
  input: {
    backgroundColor: "#fff",
    borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8,
    paddingHorizontal: 12, paddingVertical: 10, fontSize: 14,
  },
  // textAlignVertical:"top" : 複数行で上揃えにする
  textarea: { height: 100, textAlignVertical: "top" },
  // ステータスボタンを横並びに
  statusRow: { flexDirection: "row", gap: 8 },
  statusButton: {
    flex: 1, alignItems: "center", paddingVertical: 10,
    borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8, backgroundColor: "#fff",
  },
  // 選択中の強調
  statusButtonActive: { borderColor: "#1e40af", backgroundColor: "#eff6ff" },
  statusText: { fontSize: 13, color: "#475569" },
  statusTextActive: { color: "#1e40af", fontWeight: "700" },
  saveButton: {
    backgroundColor: "#1e40af", paddingVertical: 14, borderRadius: 10, alignItems: "center", marginTop: 8,
  },
  // 保存中はグレーにして押せない雰囲気に
  saveButtonDisabled: { backgroundColor: "#94a3b8" },
  saveButtonText: { color: "#fff", fontWeight: "bold", fontSize: 15 },
});
```

3.1 重要ポイントの解説

① **入力チェック（バリデーション）** `title.trim() === ""` で「タイトルが空（または空白だけ）」かを判定し、必須項目が未入力なら `Alert.alert` で警告を出して保存を中断します。`trim()` は前後の空白を取り除くので、「スペースだけ入力」も未入力扱いにできます。

> **`Alert.alert` とは？** スマホOS標準の「ポップアップ警告」を出す関数です。`Alert.alert("タイトル", "本文")` の形で、ユーザーに「OK」を押させるダイアログを表示します。入力ミスや確認メッセージに使います。

② **二重送信の防止（`saving` state）** 保存ボタンを連打されると、同じ本が何度も登録されてしまいます。`saving` というstateで「保存処理中」を管理し、その間はボタンを `disabled`（押せない）にし、文言を「保存中...」に変えます。これはアプリ品質を上げる定番テクニックです。

③ **`style` に配列を渡す** `style={[styles.statusButton, status === option && styles.statusButtonActive]}` のように `style` に**配列**を渡すと、複数のスタイルを重ねられます。`条件 && スタイル` は「条件がtrueのときだけそのスタイルを適用」という意味で、選択中のボタンだけを強調するのに使っています。

> **`条件 && 値` の意味:** `&&`（アンド）を使った `status === option && styles.statusButtonActive` は、「左がtrueなら右の値、falseなら何もしない（false）」という短い書き方です。Reactでは「条件を満たすときだけ何かを表示／適用する」のに多用します。

④ **`onChangeText={setTitle}` の短縮** 第4章では `onChangeText={(text) => setTitle(text)}` と書きましたが、これは `onChangeText={setTitle}` と短く書けます。「受け取った文字をそのまま `setTitle` に渡す」だけなので、関数を直接指定できるのです。

▼ コードを1つずつ分解して解説

上の `app/new.tsx` も、塊ごとにもう一度コードを載せながらていねいに見ていきます。「入力 → チェック → 保存 → 戻る」という流れを、コードのどの部分が担当しているかを意識して読んでください。

解説1: import と選択肢の準備

コピー

```
import { useState } from "react";
import { View, Text, TextInput, Pressable, StyleSheet, ScrollView, Alert } from "react-native";
import { useRouter } from "expo-router";
// 第6章で作った「追加」関数
import { createBook } from "../lib/books";

// ステータスの選択肢を配列で用意（ボタンを並べるのに使う）
const STATUS_OPTIONS = ["未読", "読書中", "読了"];
```

- 1〜4行目で必要な道具を読み込みます。新しく出てくるのは `TextInput`（入力欄）、`ScrollView`（スクロールできる箱）、`Alert`（警告ダイアログ）、`createBook`（第6章で作った追加関数）です。
- `STATUS_OPTIONS` は「未読・読書中・読了」という選択肢を入れた配列です。これを後で `map` でボタンに変換します。
- この配列をコンポーネントの**外側**に置いているのは、毎回作り直す必要のない固定値だからです。

> **用語: TextInput（テキストインプット）** … React Native の「文字を入力する欄」の部品。Webの `<input>` にあたります。`value` と `onChangeText` を組み合わせて使います。

解説2: 入力欄ごとに state を用意する

コピー

```
const router = useRouter();

// 入力欄ごとにstateを用意する（第4章の制御コンポーネント）
// タイトル（最初は空文字）
const [title, setTitle] = useState("");
// 著者
const [author, setAuthor] = useState("");
// ステータス（初期値は未読）
const [status, setStatus] = useState("未読");
// メモ
const [memo, setMemo] = useState("");
// 保存処理中かどうか（連打防止に使う）
const [saving, setSaving] = useState(false);
```

- 入力欄1つにつき1つの state を用意し、「今その欄に何が入力されているか」を覚えておきます。
- `title` / `author` / `memo` は最初は空文字 `""`、`status` は最初から `"未読"` を選んだ状態にしています。
- `saving` は「いま保存処理中か」を表す `true` / `false` で、連打による二重登録を防ぐのに使います（解説4・解説7で活躍します）。

> **用語: 制御コンポーネント（controlled component）** … 入力欄の値を state で管理し、`value={state}` と `onChangeText={set...}` で結びつける書き方。入力内容を React が常に把握でき、チェックや加工がしやすくなります。

解説3: 保存処理 `handleSave`（入力チェック）

コピー

```
const handleSave = async () => {
  // 入力チェック（バリデーション）：タイトルと著者は必須
  // .trim() ：文字列の前後の空白を除く。空白だけの入力を「未入力」とみなすため
  if (title.trim() === "" || author.trim() === "") {
    // Alert.alert(タイトル, 本文) ：警告ダイアログを出す
    Alert.alert("入力エラー", "タイトルと著者は必須です");
    // 処理を中断（保存しない）
    return;
  }
```

- `handleSave` は保存ボタンを押したときに動く関数です。`async` が付いているのは、中で `createBook` の完了を `await`（待つ）するためです。
- まず**入力チェック（バリデーション）**を行います。`title.trim() === ""` は「タイトルが空、または空白だけ」かの判定です。`.trim()` で前後の空白を取り除くので、スペースだけの入力も「未入力」とみなせます。

- `||`（または）でタイトルか著者のどちらかが空ならエラー扱いにし、`Alert.alert` で警告を出して `return` で処理を中断します（保存しません）。

> **用語: バリデーション（validation）** … 入力内容が正しいか（必須項目が埋まっているか等）を保存前に確認すること。不正なデータがサーバーに送られるのを防ぎます。

解説4: 保存処理 `handleSave`（保存と画面遷移）

コピー

```
  try {
    // 保存開始（ボタンを無効化して二重送信を防ぐ）
    setSaving(true);
    // createBook(...) : 第6章の追加関数。入力値をオブジェクトにまとめて渡す
    await createBook({
      // 前後空白を除いて保存
      title: title.trim(),
      author: author.trim(),
      status: status,
      // メモが空ならnull、あれば値を保存（三項演算子）
      memo: memo.trim() === "" ? null : memo.trim(),
    });
    // 成功したら一覧画面へ戻る。一覧側のuseFocusEffectが再取得し、追加した本が表示される
    router.back();
  } catch (e) {
    Alert.alert("保存に失敗しました", "通信状況を確認してもう一度お試しください");
    console.log("保存エラー:", e);
  } finally {
    // 成功・失敗どちらでも保存状態を解除
    setSaving(false);
  }
};
```

- チェックを通過したら `setSaving(true)` で「保存中」に切り替えます。これでボタンが押せなくなり、二重登録を防げます。
- `await createBook({...})` で、入力値を1つのオブジェクトにまとめてSupabaseに保存します。各値は `.trim()` で前後の空白を除いてから渡します。
- `memo.trim() === "" ? null : memo.trim()` は**三項演算子**で、「メモが空なら `null`、あれば入力値」を保存する意味です。
- 成功したら `router.back()` で一覧画面へ戻ります。戻った瞬間に一覧側の `useFocusEffect` が再取得し、追加した本が表示されます。失敗時は `catch` で警告を出し、`finally` で必ず `saving` を解除します。

> **用語: 三項演算子（条件 ? A : B）** …「条件が成り立てば A、そうでなければ B」を1行で表す式。`if` 文より短く、値を返す場面で重宝します。

解説5: 入力欄の JSX（`TextInput` と制御コンポーネント）

コピー

```
{/* タイトル入力 */}
<View>
  <Text style={styles.label}>タイトル *</Text>
  <TextInput
    style={styles.input}
    placeholder="書籍のタイトルを入力"
    // stateと結びつける
    value={title}
    // 入力が変わるたびsetTitleで更新（setTitle(text)の短縮形）
    onChangeText={setTitle}
  />
</View>
```

- 全体は `ScrollView` で囲まれており、入力欄が増えてキーボードで隠れてもスクロールして見られます。
- `TextInput` が実際の入力欄です。`placeholder` は何も入力していないときに薄く表示される案内文です。
- `value={title}` で state と画面を結びつけ、`onChangeText={setTitle}` で「文字が変わるたびに `setTitle` で state を更新」します。この**値の表示と更新を state にひもづける**のが制御コンポーネントです。
- 著者・メモ欄も同じ仕組みで、それぞれ対応する state（`author` / `memo`）につながっています。

> **用語: placeholder（プレースホルダー）** … 入力欄が空のときに薄く表示される見本テキスト。「何を入力すればよいか」をユーザーに示します。入力すると消えます。

解説6: ステータス選択ボタン（`map` と条件付きスタイル）

コピー

```
{/* ステータス選択（3つのボタンから1つ選ぶ） */}
<View>
  <Text style={styles.label}>ステータス</Text>
  <View style={styles.statusRow}>
    {/* STATUS_OPTIONS配列をmapでボタンに変換（第2章のmap） */}
    {STATUS_OPTIONS.map((option) => (
      <Pressable
        // map で並べる要素には必ずkeyが必要
        key={option}
        // style に配列を渡すと複数スタイルを合成できる。選択中なら強調スタイルを追加
        style={[styles.statusButton, status === option && styles.statusButtonActive]}
        // 押したらそのステータスを選択
        onPress={() => setStatus(option)}
      >
        <Text style={[styles.statusText, status === option && styles.statusTextActive]}>
          {option}
        </Text>
      </Pressable>
    ))}
  </View>
</View>
```

- `STATUS_OPTIONS.map((option) => (...))` で、配列の各文字（未読・読書中・読了）を1つずつボタン（`Pressable`）に変換して並べています。
- `key={option}` は `map` で並べる要素に**必ず付ける目印**です。React が各ボタンを区別するために使います。
- `style={[styles.statusButton, status === option && styles.statusButtonActive]}` は、`style` に**配列**を渡して複数スタイルを重ねる書き方です。`status === option && スタイル` は「今選ばれている選択肢のときだけ強調スタイルを足す」という意味です。
- `onPress={() => setStatus(option)}` で、押されたボタンの選択肢を `status` に保存します。これで選択中のボタンだけ色が変わります。

> **用語: 条件付きスタイル（`条件 && スタイル`）** … `&&` を使い「条件が成り立つときだけそのスタイルを適用する」テクニック。選択中・エラー中など、状態に応じた見た目の変化に使います。

解説7: 保存ボタン（二重送信の防止）

コピー

```
{/* メモ入力（複数行） */}
<View>
  <Text style={styles.label}>メモ</Text>
  <TextInput
    style={[styles.input, styles.textarea]}
    placeholder="感想や覚え書きなど"
    value={memo}
    onChangeText={setMemo}
    // multiline : 複数行入力を許可する
    multiline
    // 表示上の高さの目安（4行分）
    numberOfLines={4}
  />
</View>

{/* 保存ボタン。saving中は無効化＆文言変更 */}
<Pressable
  style={[styles.saveButton, saving && styles.saveButtonDisabled]}
  onPress={handleSave}
  // disabled : trueの間は押せなくする（二重送信防止）
  disabled={saving}
>
  <Text style={styles.saveButtonText}>{saving ? "保存中..." : "保存する"}</Text>
</Pressable>
```

- メモ欄の `TextInput` には `multiline` を付けて複数行入力を許可し、`numberOfLines={4}` で高さの目安を4行分にしています。
- 保存ボタンは `onPress={handleSave}` で解説3・4の保存処理を呼び出します。
- `disabled={saving}` は「`saving` が `true` の間はボタンを押せなくする」設定で、これが**二重送信の防止**の本体です。`style` 側でも `saving && styles.saveButtonDisabled` でグレーに変え、押せない雰囲気を出します。
- `{saving ? "保存中..." : "保存する"}` は三項演算子で、保存中はボタンの文字を「保存中...」に変えてユーザーに状況を伝えます。

> **用語: disabled（ディスエーブルド）** … ボタンを「押せない状態」にする設定。保存処理中に `true` にすることで、連打による同じデータの二重登録を防ぎます。

解説8: `styles`（見た目の定義）

コピー

```
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f8fafc" },
  label: { fontSize: 13, fontWeight: "600", color: "#334155", marginBottom: 6 },
  input: {
    backgroundColor: "#fff",
    borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8,
    paddingHorizontal: 12, paddingVertical: 10, fontSize: 14,
  },
  // textAlignVertical:"top" : 複数行で上揃えにする
  textarea: { height: 100, textAlignVertical: "top" },
  // ステータスボタンを横並びに
  statusRow: { flexDirection: "row", gap: 8 },
  statusButton: {
    flex: 1, alignItems: "center", paddingVertical: 10,
    borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8, backgroundColor: "#fff",
  },
  // 選択中の強調
  statusButtonActive: { borderColor: "#1e40af", backgroundColor: "#eff6ff" },
  statusText: { fontSize: 13, color: "#475569" },
  statusTextActive: { color: "#1e40af", fontWeight: "700" },
  saveButton: {
    backgroundColor: "#1e40af", paddingVertical: 14, borderRadius: 10, alignItems: "center", marginTop: 8,
  },
  // 保存中はグレーにして押せない雰囲気に
  saveButtonDisabled: { backgroundColor: "#94a3b8" },
  saveButtonText: { color: "#fff", fontWeight: "bold", fontSize: 15 },
});
```

- 入力欄・ラベル・ボタンなどの見た目を `StyleSheet.create` でまとめて定義しています。
- `statusRow` の `flexDirection: "row"` は「中身を横並びにする」設定で、3つのステータスボタンを横一列に並べます。`gap` はボタン同士のすき間です。
- `statusButtonActive` と `statusTextActive` は、解説6の条件付きスタイルで「選択中だけ」追加されるスタイルです（枠線と文字色を青で強調）。
- `saveButtonDisabled` は、解説7で保存中だけ追加されるスタイルで、ボタンをグレーにして「いま押せない」ことを見た目で伝えます。

> **用語: flexDirection（フレックスディレクション）** … 子要素を「縦並び（`column`、初期値）」か「横並び（`row`）」のどちらに並べるかを決める設定。`row` でボタンを横一列にできます。

---

4. 動作確認

`npx expo start` でアプリを起動し、次を確認します。

1. 一覧画面に、第5章で入れたテストデータ（3冊）が表示される。
2. 右下の「＋」を押すと登録画面へ遷移する。
3. タイトル・著者を空のまま保存 →「入力エラー」の警告が出る。

4. タイトル・著者・ステータスを入力して「保存する」→ 一覧画面に戻り、**追加した本が一番上に表示される**（`created_at` の降順で並べているため）。
5. Supabaseのダッシュボード（Table Editor）でも、追加した行が増えているのを確認できる。

> **よくあるつまずき:** - 一覧が空のまま → 第5章のRLSポリシー設定を確認（未設定だとデータが読めない）。- 追加しても一覧に出ない → `index.tsx` が `useEffect` でなく `useFocusEffect` になっているか確認。- 保存でエラー → `lib/books.ts` の `createBook` と、`.env` の接続情報を確認。

---

5. この章のまとめ

- Read: `fetchBooks()` で全件取得し、`FlatList` で一覧表示。`ActivityIndicator` で読込中、`ListEmptyComponent` で0件時を表示
- `useFocusEffect` を使い、登録画面から戻った瞬間に最新データを再取得
- `try / catch / finally` で通信エラーに備える
- Create: `TextInput` の入力をstateで管理し、`createBook()` で登録。**バリデーション**と**二重送信防止**も実装
- `Alert.alert` でユーザーへの通知、`style` 配列＋ `&&` で条件付きスタイル

> **次の章へ:** 本を「表示」「追加」できるようになりました。第8章では残りの **U（Update＝編集）** と **D（Delete＝削除）** を実装し、CRUDを完成させます。